



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 3

Team No.: 5

Project Title: University Laboratory Management Server

Team Members

Roll Number	Student Name	Signature
2520030022	A V S N R Akhila	<i>Akhila</i>
2520030098	G. Sushma Sree	<i>Sushma</i>
2520030335	Manasvi Nilesh Ghag	<i>Manasvi</i>

1. Abstract

The University Computer Laboratory Management Server is a Linux-based system that manages requests from many students at the same time. Students can request resources, submit programs, access files, and perform lab activities through the server.

The system uses worker threads to handle multiple requests, mutexes to protect shared data, and semaphores to control limited resources. It also uses signals for server shutdown, monitors processes and threads, keeps activity logs, and shows resource usage and availability.

2. Problem Statement

A university computer laboratory contains a large number of computer systems and limited shared resources that are accessed by multiple students simultaneously. Students may submit programs, request system resources, access files, and perform other laboratory activities at the same time.

Without a centralized management mechanism, simultaneous requests can result in resource conflicts, race conditions, inefficient resource utilization, and difficulty in tracking active activities.

Therefore, a centralized Linux-based laboratory management server is required to accept and process multiple student requests concurrently, synchronize access to shared data, control limited resources, maintain request and allocation information, monitor processes and threads, and provide server activity and resource-availability information.

3. Objectives

1. Handle multiple student requests at the same time.
2. Use worker threads to process requests.
3. Maintain a shared request queue.
4. Use mutexes to safely access shared data.
5. Use semaphores to control limited resources.
6. Track student requests and resource allocation.
7. Use signals for server control and shutdown.
8. Monitor processes, threads, and resource usage.
9. Maintain server activity logs.

4. Proposed Methodology

The server will be developed in C on Linux. Clients send requests to the server, which places them in a shared queue. Worker threads take requests from the queue and process them.

Mutexes will protect the shared queue and data, while semaphores will control access to limited resources. The server will also record activities in logs, monitor active threads/processes, display resource availability, and use signals for safe shutdown.

Working Flow

Student Client → Server → Request Queue → Worker Thread → Resource Allocation → Processing → Response → Logging

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
POSIX Threads	Handle multiple requests
Mutex	Protect shared data
Semaphores	Control limited resources
Signals	Server control and shutdown
File I/O	Maintain logs
Process Management	Monitor processes
Synchronization	Prevent conflicts/race conditions

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030022	A V S N R Akhila	Implement the Linux server process structure, process creation and management, request handling, process monitoring, and server control using POSIX system calls. Handle process lifecycle, signals, process synchronization, and server shutdown.
2520030098	Gutha Sushma Sree	Implement POSIX threads for handling multiple student requests concurrently. Manage the shared request queue using mutexes and condition variables, and use counting semaphores to control access to limited laboratory resources while preventing race conditions and concurrency problems.
2520030335	Manasvi Nilesh Ghag	Implement laboratory activity and transaction logs using Linux file I/O and file descriptors. Handle

		resource/request data storage and file operations, and apply memory management concepts such as dynamic memory allocation and memory-mapped file I/O where required.
--	--	--

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system
C / C++	Programming language
POSIX Libraries	Threads and semaphores
GCC	Compile the program
VS Code	Coding

8. Expected Outcome

The project will produce a working Linux-based laboratory management server that can handle multiple student requests simultaneously, manage limited resources safely, and track requests and activities.

It will demonstrate important OS concepts such as threads, synchronization, semaphores, mutexes, signals, process management, resource management, and client-server communication.

9. GitHub Repository

GitHub Repository Link: <https://github.com/guthasushmasree/OSSP--University-Laboratory-Management-Server>

Repository Created: Yes

☒ Yes ☐ No

Faculty Name and Signature

Faculty Name: K. Hema

Signature: _____

Remarks:

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____